

Hash Join Optimization

Muyan Zhou

June 22, 2025

1 Introduction

This project explores how to optimize hash joins using Yannakakis algorithm with Bloom filters. When applying the standard approach to binary join sequences, I found it ran 3-5 times slower than regular hash joins. Through several optimizations like batch processing and join order adjustment, I managed to reduce the slowdown to just 1-1.5 times. The results show interesting trade-offs between theory and practice in query processing.

2 Optimization

2.1 Predicate Transfer

The Yannakakis algorithm efficiently evaluates acyclic join queries by first transforming the join graph into a rooted tree structure, then performing forward and backward passes to eliminate non-matching tuples from the tables, and finally joining the reduced tables to produce the result, achieving an optimal time complexity of $O(N + OUT)$ where N is the input size and OUT is the output size. An enhanced version called Predicate Transfer further optimizes this process by replacing the semi-joins in both passes with more efficient Bloom filters, significantly accelerating the computation.

In this project, the input consists of a standard binary join sequence rather than a join graph. To address this, I first reconstruct the join graph (which forms a tree structure), select a root table, and then apply the Predicate Transfer optimization. Initially, I perform a full scan of all tables to collect their tuples. For each semi-join operation $R \bowtie S$, where S acts as a filter for R , I construct a Bloom filter B_S for S , scan R sequentially to retain only the tuples matching B_S , and replace R with this filtered version. Surprisingly, this approach performs poorly—running 3–5 times slower than the baseline implementation. Consequently, I explore several optimizations to improve its efficiency.

2.1.1 Vectorized Model

Initially, scanning all tables upfront proved prohibitively expensive. To address this, I adopted a vectorized execution model by replacing the full-table materi-

alization (via `ValueOperator`) with a streaming `SeqScanOperator`. Instead of loading entire tables into memory, this approach incrementally fetches batches of tuples through `Next()`, significantly reducing overhead. While this optimization narrowed the performance gap, the implementation remained 2–3 times slower than the baseline, prompting further refinements.

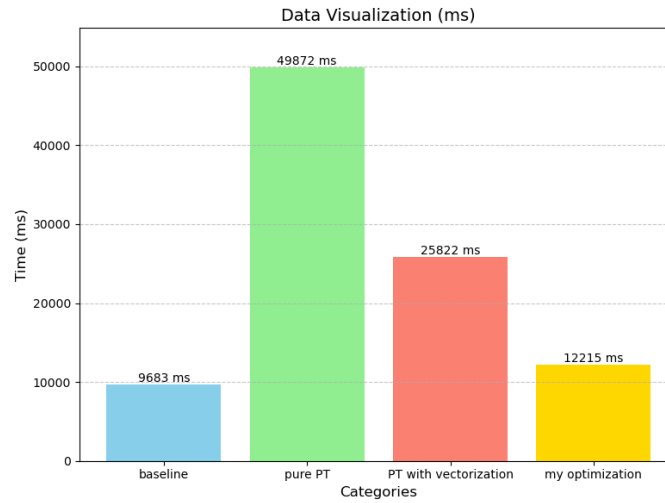
2.1.2 Linked List

In each semi-join $R \ltimes S$, the goal is to filter unmatched tuples from R using a Bloom filter B_S (constructed from S). The initial implementation materialized all matched tuples—even though unmatched entries could safely be ignored—introducing unnecessary overhead. To optimize this, I replaced the materialization step with an in-place deletion mechanism using a linked list to track active tuples, implemented via a `JumpScanOperator`. This operator scans R while skipping deleted entries, avoiding redundant storage. Surprisingly, the overhead of maintaining the linked list nearly negated the gains from vectorization, resulting in negligible net improvement. I suspect this is due to the method’s poor cache locality, as the linked list’s non-contiguous memory access patterns likely caused frequent cache misses, undermining the potential benefits of vectorization.

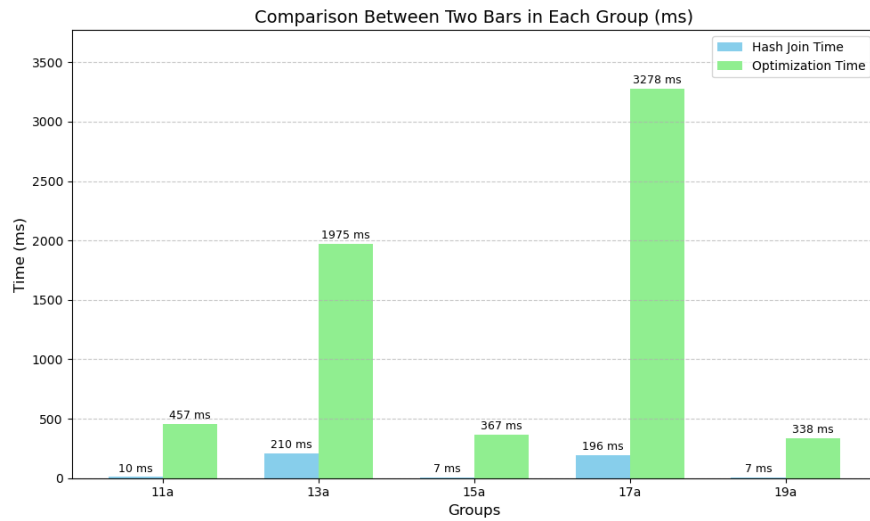
2.2 Other Optimizations

During semi-join operations ($R \ltimes S$), S is often larger than R —particularly during the forward pass—making the traditional two-pass approach inefficient. To address this, I abandoned both the forward and backward passes entirely, instead executing semi-joins directly along the predefined binary join sequence. This adjustment reduced the overhead significantly, narrowing the performance gap to just 1–1.5× slower than the baseline.

3 Evaluation and Comment



The figure above shows the total time the algorithm costs in four different cases.



This figure shows the effects of optimization more clearly. After the prepare time(a.k.a. optimization time), the actual run time, namely the hash join time, is really short.